

Natural Language Processing

Lecture 5 Discriminative Learning

The Perceptron,
Linear Classifiers,
& Architecture

Where Are We Now?



LECTURE 1 Introduction to NLP



LECTURE 2 Learning, Search & Representations



LECTURE 3 BoW, TF-IDF & Vector Space Models



LECTURE 4 Naïve Bayes & Smoothing



TODAY **Discriminative Learning** » **Perceptron**

Learning Objectives



Explain generative vs discriminative learning.



Explain the Perceptron model architecture.



Apply the Perceptron prediction rule.



Understand convergence and limitations.



Understand linear classification & boundaries.



Understand the role of weights and bias.



Perform manual weight updates.



Apply Perceptron to text classification.

Two Ways to Classify Text

"This movie is fantastic."

APPROACH 1: GENERATIVE

"How likely is this sentence under each class?"

Example: Naïve Bayes

APPROACH 2: DISCRIMINATIVE

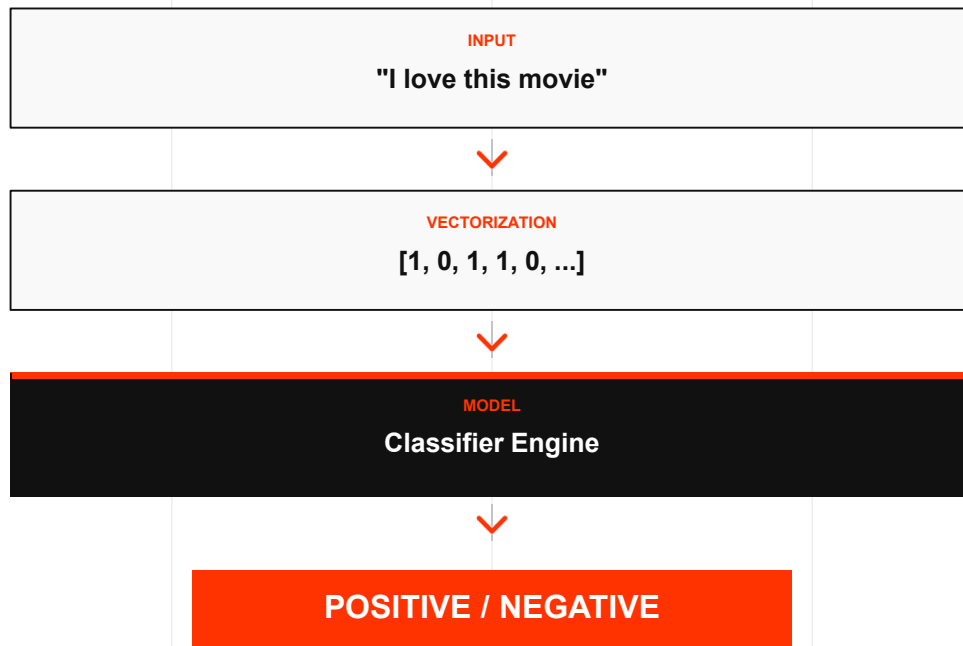
"Where should the decision boundary be placed?"

Example: Perceptron / Logistic Regression

Generative vs Discriminative

FEATURE	Generative Models	Discriminative Models
FOCUS	Models data distribution $P(X,Y)$	Models decision boundary $P(Y X)$
GOAL	Learns how data is generated	Learns a decision function
EXAMPLE	Naïve Bayes	Perceptron
ASSUMPTIONS	Strong probability assumptions	Direct classification focus

The Classification Problem



What Is a Linear Classifier?

$$\text{Score} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

x

Input Features (e.g., word counts or presence in a document).

w

Learned Weights representing the importance of each feature.

b

Bias term that shifts the decision boundary from the origin.

Intuition Behind Weights

WORD FEATURE

LEARNED WEIGHT

excellent

+3.0

fantastic

+2.5

good

+1.5

bad

-1.5

terrible

-3.0

POSITIVE WEIGHTS

Associated with the positive class. They push the final score **upward**.

NEGATIVE WEIGHTS

Associated with the negative class. They push the final score **downward**.

Decision Boundary

Score > 0

PREDICT POSITIVE (+1)

Score < 0

PREDICT NEGATIVE (-1)

THE BOUNDARY

The decision boundary occurs exactly when **Score** = **0**.

GEOMETRIC INTERPRETATION

In 2D, this is a **Line**. In higher dimensions, it becomes a **Hyperplane**.

Visual Intuition of Linear Separation

THE HYPERPLANE

In two dimensions, the decision boundary is a **line**.

In higher dimensions (like NLP with thousands of words), it becomes a **hyperplane** that separates the feature space.

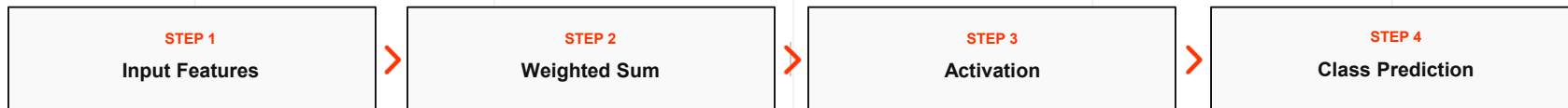


What Is the Perceptron?

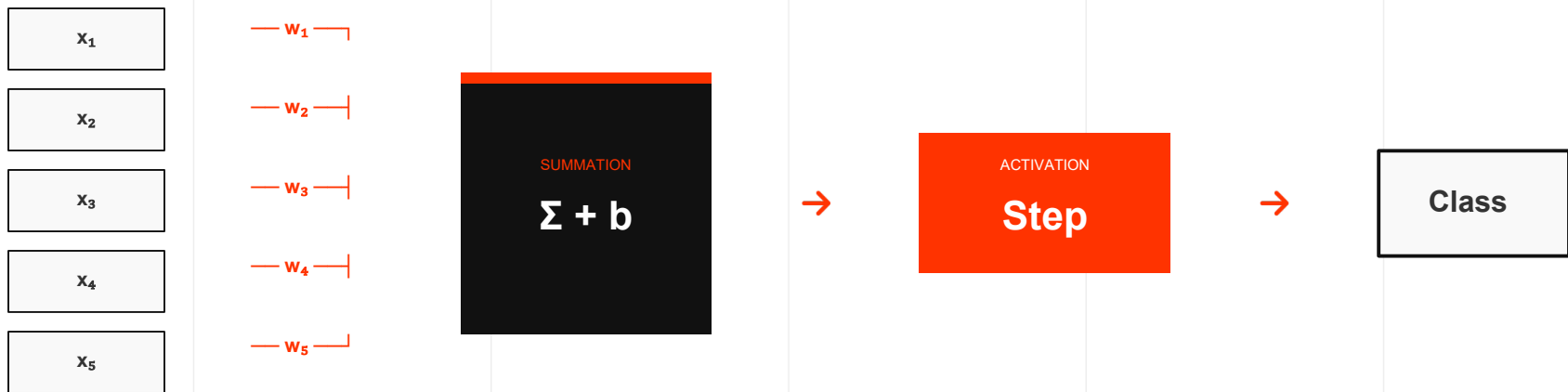
Frank Rosenblatt

INTRODUCED IN 1957

The Perceptron is one of the earliest neural-network-inspired learning algorithms, designed to learn a linear decision boundary for binary classification.



Perceptron Architecture



Each feature contributes to the final decision according to its **learned weight**. The bias term **b** provides the necessary offset for the decision boundary.

Perceptron Prediction Rule

STEP 1: CALCULATE SCORE

$$z = w \cdot x + b$$

IF $z \geq 0$

Predict $+1$ (Positive)

IF $z < 0$

Predict -1 (Negative)

Numerical Example: Prediction

1. GIVEN INPUTS

$x = [2, 3]$

$w = [1, -1]$

$b = 0$

2. CALCULATION

$$\begin{aligned}z &= (1)(2) + (-1)(3) \\z &= 2 - 3 \\z &= -1\end{aligned}$$

3. FINAL RESULT

Since $z < 0$, the step function outputs:

Prediction = -1

How Does the Perceptron Learn?

- 01** **INITIALIZATION**
Start with weights as random or zero.
- 02** **OBSERVATION**
Take one training example (x, y) .
- 03** **PREDICTION**
Use current weights to predict class.
- 04** **COMPARISON**
Compare prediction with the true label (y) .
- 05** **UPDATE**
If wrong, update weights; otherwise, keep them.
- 06** **ITERATION**
Repeat for all examples in the dataset.

Perceptron Update Rule

$$\begin{aligned}w_{\text{new}} &= w_{\text{old}} + \eta y x \\b_{\text{new}} &= b_{\text{old}} + \eta y\end{aligned}$$

η (ETA)

Learning Rate determines the step size of the update.

y

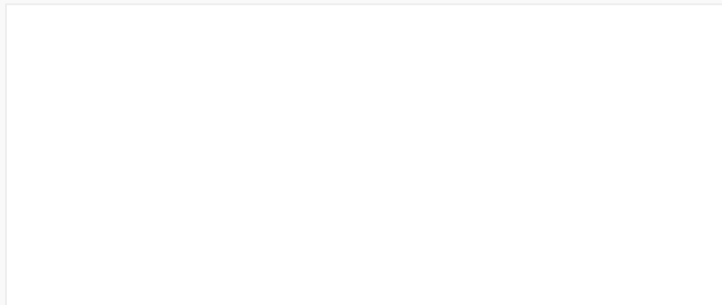
True Class label (+1 or -1) of the training example.

x

Feature Vector of the misclassified example.

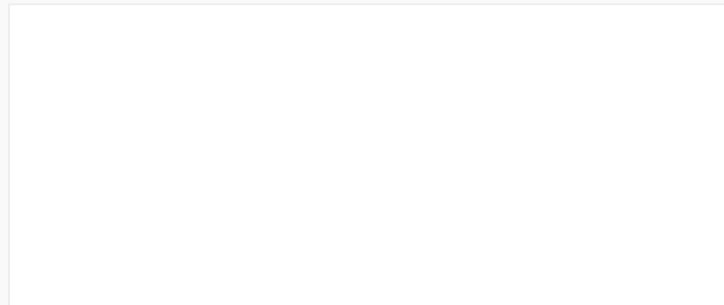
Understanding the Update Logic

IF $Y = +1$ (PREDICTED -1)



The update $\mathbf{w} + \eta \mathbf{x}$ moves the boundary closer to the positive class example, increasing the likelihood of a positive score.

IF $Y = -1$ (PREDICTED $+1$)



The update $\mathbf{w} - \eta \mathbf{x}$ moves the boundary away from the negative class example, decreasing the score.

The boundary effectively "rotates" and "shifts" to fix its mistakes.

Worked Example: Weight Update

1. INITIAL STATE

$x = [2, 1]$
 $y = +1$ (True Class)
 $w = [0, 0]$
 $b = 0$
 $\eta = 1$

2. UPDATE CALCULATION

$w_{\text{new}} = w_{\text{old}} + \eta y x$
 $w_{\text{new}} = [0, 0] + 1(+1)[2, 1]$

$b_{\text{new}} = b_{\text{old}} + \eta y$
 $b_{\text{new}} = 0 + 1(+1)$

Applying the Perceptron Rule to fix a misclassification.

3. UPDATED STATE

$w_{\text{new}} = [2, 1]$
 $b_{\text{new}} = 1$

New Boundary Set

Worked Example: Testing the Model

CURRENT MODEL STATE

$w = [2, 1]$

$b = 1$

NEW INPUT

$x = [1, 2]$

CALCULATION

$z = w \cdot x + b$

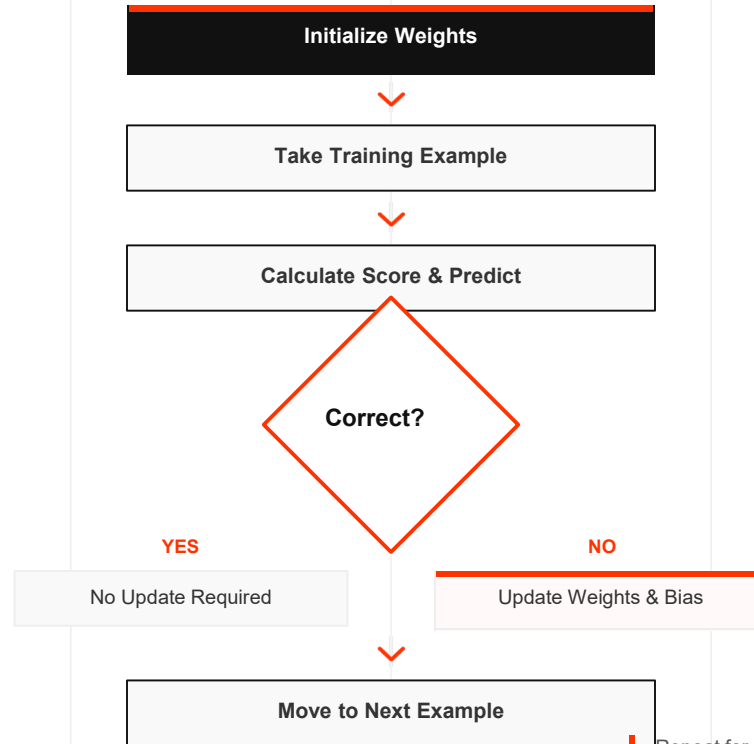
$z = 2(1) + 1(2) + 1$

$z = 2 + 2 + 1$

$z = 5$

PREDICTION = +1

Perceptron Training Loop

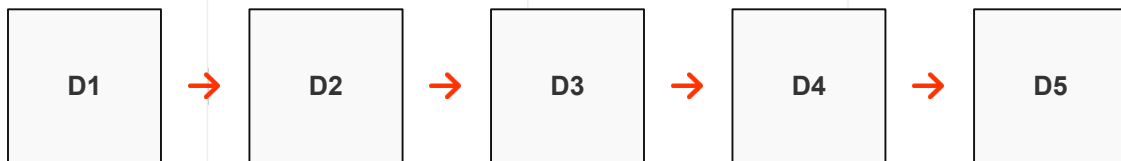


Repeat for multiple **Epochs** until convergence.

What Is an Epoch?

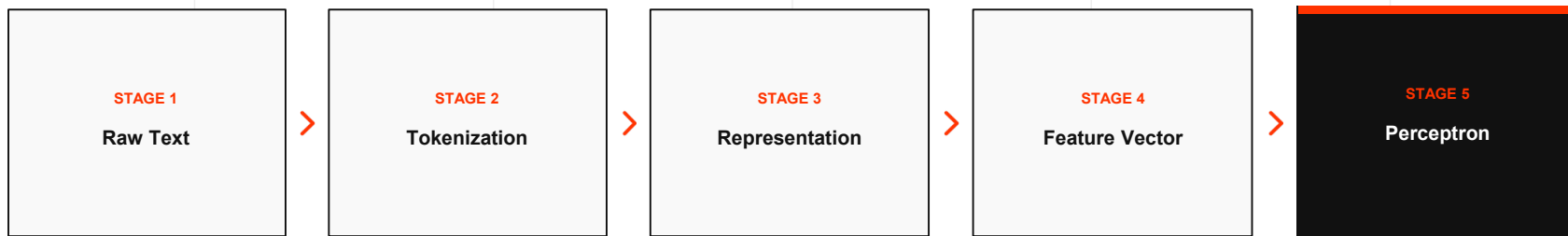
DEFINITION

An Epoch is one complete pass through the entire training dataset.



After processing all examples (D1 to D5), **one epoch** is complete. The model then begins the next epoch, often shuffling the data to improve learning.

Perceptron in the NLP Pipeline



Example: "This movie is excellent." → [0, 1, 0, 1, 1, ...] → Positive

Example: Sentiment Classification

1. SETUP

Vocabulary:

{good, bad, excellent, terrible}

Sentence:

"excellent good"

Vector (x):

[1, 0, 1, 0]

2. CALCULATION

Weights (w):

[1, -2, 3, -3]

Score (z):

$$\begin{aligned}z &= 1(1) + (-2)(0) + 3(1) + (-3)(0) \\z &= 1 + 0 + 3 + 0 \\z &= 4\end{aligned}$$

3. PREDICTION

Since $z > 0$, the model predicts:

POSITIVE

The positive weights for "good" and "excellent" combined to push the score above zero.

Why Does It Work?

POSITIVE ASSOCIATION

- + excellent
- + wonderful

Positive weights push the final score toward the positive class boundary.

NEGATIVE ASSOCIATION

- terrible
- boring

Negative weights push the final score toward the negative class boundary.

Weights encode the "sentiment polarity" of features.

Strengths of the Perceptron



SIMPLICITY

Extremely easy to implement and understand.



EFFICIENCY

Very fast training and prediction times.



HIGH DIMENSIONS

Handles sparse features (like BoW) exceptionally well.



LINEAR BASELINES

Provides a strong starting point for classification.



FOUNDATION

A critical historical stepping stone toward deep learning.

Limitations: The Linear Constraint

LINEAR SEPARABILITY

The Perceptron can only solve problems where classes can be separated by a single straight line or hyperplane.

The classic **XOR Problem**:

No single straight line can perfectly separate the black dots from the grey circles. The Perceptron will **never converge** on this data.

Perceptron vs Naïve Bayes

FEATURE	Naïve Bayes	Perceptron
TYPE	Generative	Discriminative
OUTPUT	Probabilities	Hard Labels (+1 / -1)
TRAINING	Counting / Frequencies	Iterative Updates
ASSUMPTION	Conditional Independence	Linear Separability

Perceptron vs Logistic Regression

PERCEPTRON

$$y = \text{sign}(w \cdot x + b)$$

- **Output:** Hard decisions (+1 or -1).
- **Optimization:** Simple error-driven updates.
- **Softness:** None; rigid decision boundary.

LOGISTIC REGRESSION

$$P(y|x) = \sigma(w \cdot x + b)$$

- **Output:** Probabilities (0 to 1).
- **Optimization:** Gradient Descent on Log Loss.
- **Softness:** Probabilistic; better generalization.

Classroom Exercise: Manual Update

GIVEN DATA

$x = [2, 1]$

$y = +1$

$w = [0, 0]$

$b = 0$

$\eta = 1$

01 Calculate the initial score (z).

02 Determine the model's prediction.

03 Perform the weight and bias update.

04 Verify prediction with new parameters.

Work in pairs. You have **5 minutes** to complete the calculation.

Summary & Next Lecture

TODAY'S RECAP

- ✓ Generative vs Discriminative models.
- ✓ Perceptron architecture and learning rules.
- ✓ The role of epochs and weights in text classification.
- ✓ Strengths and limitations (linear separability).

COMING UP NEXT

Logistic Regression

- Moving from hard labels to probabilities.
- The Sigmoid Activation Function.
- Optimization via Gradient Descent.